

DAY 18: DATA ANALYTICS MASTER CLASS

Module: One-Sample T-Test (Fully Automated & Judged Coding Guide)



1. THE CORE PHILOSOPHY & BACKGROUND

In advanced enterprise analytics, raw calculation results must always feed into automated decision blocks to generate instant industrial actions without requiring manual intervention.

We evaluate the absolute value of our T-Score against the benchmark boundary of 2.0 multiples to decide whether to accept or reject the system stability baseline.



2. THE COMPLETE MANUAL REASONING CODE WITH VERDICT LOGIC

This full script contains both the step-by-step mathematical translation loop and the conditional logical tree used to automatically output the execution verdict.

python_complete_manual_t_test.py

```
# 1. Target Dataset Setup
coins_data = [9.8, 10.1, 9.9, 10.2, 9.7, 10.1, 9.9, 10.0, 9.8, 10.1]
target_mu = 10.0
N = len(coins_data)

# STEP A: Mean Processing
sample_mean = sum(coins_data) / N

# STEP B: Deviation Processing (N - 1)
squared_diff_sum = 0
for x in coins_data:
    squared_diff_sum += ((x - sample_mean) ** 2)
sample_std_dev = (squared_diff_sum / (N - 1)) ** 0.5

# STEP C: Error Scaling Factor
standard_error = sample_std_dev / (N ** 0.5)

# STEP D: Final T-Score Result
t_score = (sample_mean - target_mu) / standard_error
print(f"Calculated T-Score: {t_score:.4f}")

print("\n--- AUTOMATED INDUSTRIAL JUDGMENT ---")
# 2. Automated Logical Judging Tree
if abs(t_score) <= 2.0:
    print("Verdict: NULL HYPOTHESIS (H0) WON!")
    print("Action: The variance is a safe natural fluke. No machine tuning needed.")
else:
    print("Verdict: REJECT H0! CRITICAL FAILURE DETECTED!")
    print("Action: Fluctuation exceeds 2 multiples of error. Inspect production engine immediately.")
```

MANUAL JUDGED CODE EXPLANATION

The operational code executes standard array summary pipelines and stores the final division multiple inside the designated `t\_score` space.

The statement `if abs(t\_score) <= 2.0:` strips away negative sign blocks to test the pure structural scale magnitude against the statistical threshold.

If the calculated output falls within 2.0 multiples of error, the conditional block dynamically prints that the system baseline holds strong and logs an approval signature.

If the code detects a deviation outside the 2.0 safe boundary, the execution shifts to the `else` block, triggering an immediate critical factory inspection alert.

3. THE COMPLETE SHORTCUT CODE WITH STATISTICAL JUDGMENT

This professional industrial structure pairs the high-speed optimization library wrapper directly with a conditional decision gate using the alpha p-value baseline.

`python_shortcut_judged_t_test.py`

```
import scipy.stats as stats

coins_data = [9.8, 10.1, 9.9, 10.2, 9.7, 10.1, 9.9, 10.0, 9.8, 10.1]
target_mu = 10.0

# Processing via professional engineering framework
t_score, p_value = stats.ttest_1samp(coins_data, popmean=target_mu)

print(f"Industrial T-Score: {t_score:.4f}")
print(f"Statistical P-Value: {p_value:.4f}")

print("\n--- AUTOMATED INDUSTRIAL JUDGMENT ---")
# 5% Alpha significance boundary test
if p_value <= 0.05:
    print("Verdict: REJECT H0! SIGNIFICANT SHIFT DETECTED!")
    print("Action: The data modification is real. Update ledger benchmarks.")
else:
    print("Verdict: NULL HYPOTHESIS (H0) WON!")
    print("Action: Variations are statistically insignificant. System is operational.")
```

SHORTCUT JUDGED CODE EXPLANATION

The optimization method returns a secondary metric variable called `p\_value`, which maps the total probability of our piggybacked result appearing due to simple randomness.

The decision expression `if p\_value <= 0.05:` benchmarks the system output against the standard 5% significance boundary allowed in corporate data processing science.

A p-value lower than 0.05 automatically registers a structural breakthrough, directing the script to declare an absolute rejection of old baseline assumptions.

Because the clean gold coin array yields a high p-value of 0.344, the engine skips the failure trap entirely, executing the safe passage message inside the fallback segment.



4. THE FULL OUTLIER BLAST CODE WITH AUTOMATED VERDICT LOOKOUT

This simulation registers the complete execution stack of an abnormal data matrix, demonstrating how skewed variance changes the final conditional logic path.

python_outlier_judged_blast.py

```
# Array contains the 1000 grams outlier entity
blasted_data = [9.8, 10.1, 1000, 10.2, 9.7, 10.1, 9.9, 10.0, 9.8, 10.1]
target_mu = 10.0

# Analytical processing pipelines running on twisted records
mean_calc = sum(blasted_data) / len(blasted_data)
diff_sq = sum((x - mean_calc) ** 2 for x in blasted_data)
sd_calc = (diff_sq / (len(blasted_data) - 1)) ** 0.5
se_calc = sd_calc / (len(blasted_data) ** 0.5)

t_score_calc = (mean_calc - target_mu) / se_calc
print(f"Calculated Blasted T-Score: {t_score_calc:.4f}")

print("\n--- AUTOMATED INDUSTRIAL JUDGMENT ---")
# Logic block maps distorted attributes
if abs(t_score_calc) <= 2.0:
    print("Verdict: NULL HYPOTHESIS (H0) WON! (WARNING: OUTLIER BLINDNESS EFFECT)")
    print("Action: System thinks calculation is safe because Standard Error was artificially expanded!")
else:
    print("Verdict: REJECT H0! ANOMALY BLOCKED!")
```

OUTLIER BLAST JUDGMENT EXPLANATION

When the tracking logic reads the massive outlier array, the automated script evaluates a final structural T-score of exactly 0.9997 units.

Since 0.9997 is mathematically smaller than the boundary threshold of 2.0, the script passes the conditional check and executes the positive affirmation block falsely.

Strategic Conclusion: This automated code layout illustrates how data errors completely bypass simple logical evaluation blocks. It highlights the critical industrial rule that anomaly data preparation must always happen before executing automated decision logic gates.